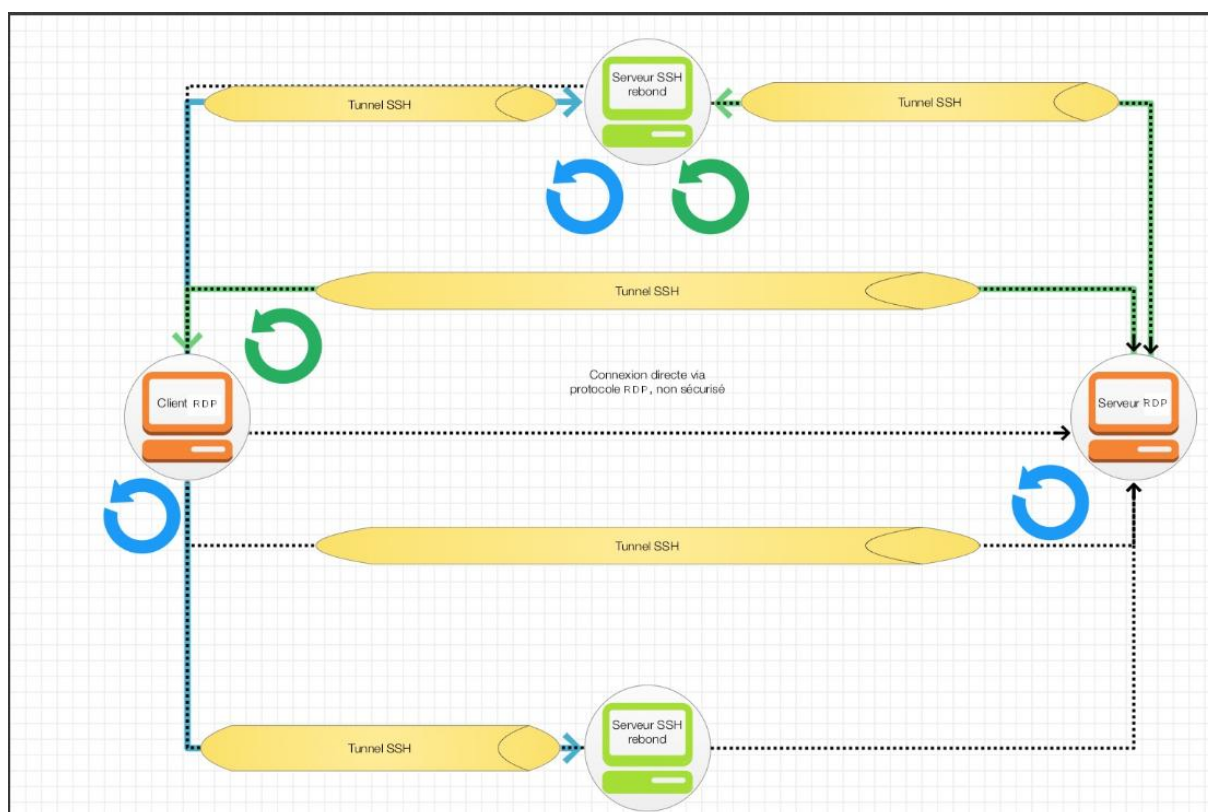


Tunnel SSH, Projet LENOBLE



Fait par Martinière Julien le 27/01/2026

Objectif du TP

L'objectif est de sécuriser une connexion Bureau à Distance (RDP) entre un client et une cible isolée en faisant converger deux tunnels SSH vers un serveur de rebond central.

Sommaire :

Etape n°1 : Connexion depuis le Client Windows	3
Etape n°2 : Remplacement du Client Windows par la Debian Clone	4
Etape n°3 : Connexion Directe par Tunnel SSH Simple.....	4
Etape n°4 : Le Double Tunnel SSH	5
1/Créations d'une clé publique et privé.....	7
2/OTP	8
3/KEEPASS	9

Etape n°1 : Connexion depuis le Client Windows

1. Architecture de départ

Dans cette première configuration, le réseau se composait de :

- **Client** : Machine physique Windows.
- **Rebond** : VM Ubuntu Server (192.168.56.103).
- **Cible** : VM Debian Graphique (192.168.56.102).

2. Procédure technique

A. Sur la Debian Cible (.102)

La préparation était identique à la phase finale : activation du partage de bureau et déconnexion de la session utilisateur pour libérer le port 3389. Le tunnel reverse était lancé vers l'Ubuntu :

```
Bash  
ssh -R 13389:127.0.0.1:3389 julien@192.168.56.103
```

Le port RDP de la Debian est ainsi "déposé" sur le port 13389 de l'Ubuntu.

B. Sur le Windows Physique (Client)

Au lieu d'utiliser une VM Linux, la connexion était initiée depuis un terminal **PowerShell** sur Windows avec un tunnel local :

```
PowerShell  
ssh -L 33389:127.0.0.1:13389 julien@192.168.56.103
```

Ce tunnel local permet de ramener le flux présent sur l'Ubuntu (port 13389) vers le port 33389 du PC Windows.

3. Connexion finale sous Windows

La validation visuelle se faisait via l'outil natif de Windows :

1. Lancement de l'application **Connexion Bureau à distance**.
2. Saisie de l'adresse de boucle locale : 127.0.0.1:33389.

3. Authentification avec les identifiants de la Debian cible (julien).

Etape n°2 : Remplacement du Client Windows par la Debian Clone

1. Objectif

L'objectif de cette phase est de supprimer la dépendance à la machine physique Windows pour simuler un réseau entièrement virtualisé et isolé. On utilise une deuxième machine Debian graphique, appelée **Debian Clone** (192.168.56.104), pour jouer le rôle de client RDP.

2. Installation des outils sur le Clone (.104)

Pour que la Debian Clone puisse remplacer Windows, il a fallu installer le client RDP **Remmina** :

- **Commande utilisée** : `sudo apt update && sudo apt install remmina remmina-plugin-rdp -y.`

Etape n°3 : Connexion Directe par Tunnel SSH Simple

1. Objectif

Avant de mettre en place le double tunnel, une étape intermédiaire consiste à créer un tunnel SSH direct (point à point) entre le **Clone (.104)** et la **Cible (.102)** pour valider la connectivité sans passer par l'Ubuntu.

2. Mise en œuvre technique

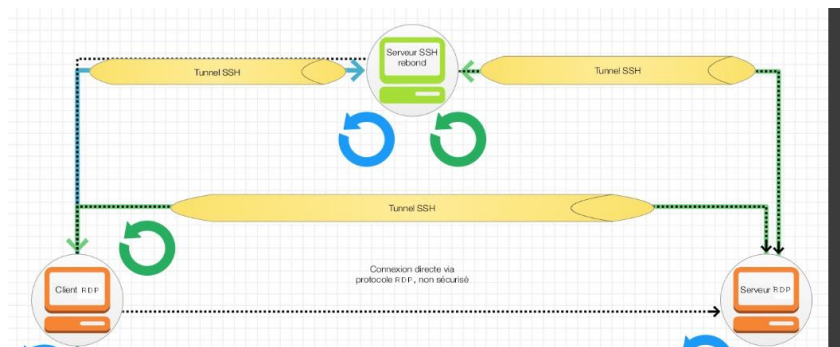
- **Sur la Debian Clone (.104)** : Lancement d'un tunnel local vers la cible.
 - **Commande** : `ssh -L 33389:127.0.0.1:3389 julien@192.168.56.102.`
- **Sur Remmina (Clone)** : Connexion à 127.0.0.1:33389.

Résultat : Cette étape prouve que les deux Debian peuvent communiquer directement, mais cette méthode n'est pas utilisée dans l'architecture finale car elle ne bénéficie pas de l'isolation offerte par le serveur de rebond.

Etape n°4 : Le Double Tunnel SSH

1. Synthèse de l'architecture finale

C'est la réalisation complète du schéma avec les deux cylindres jaunes qui se rejoignent sur l'**Ubuntu Server (.103)**.



2. Procédure de validation

- **Côté Cible (.102) :** Tunnel **Reverse** vers l'Ubuntu pour "pousser" le port RDP.

```
ssh -R 13389:127.0.0.1:3389 julien@192.168.56.103.
```

connection to 192.168.56.103 closed.

```
julien@ServeurRdp:~$ ssh -R 13389:127.0.0.1:3389 julien@192.168.56.103
```

- **Côté Client (.104) :** Tunnel **Local** vers l'Ubuntu pour "récupérer" le flux sur le port 13389.

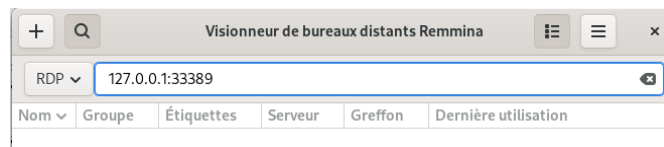
```
ssh -L 33389:127.0.0.1:13389 julien@192.168.56.103.
```

```
julien@ServeurRdp:~$ ssh -L 33389:127.0.0.1:13389 julien@192.168.56.103
```

Configuration de la connexion

Dans l'interface de Remmina sur la **Debian Clone (.104)**, les paramètres suivants ont été saisis pour s'insérer dans le tunnel :

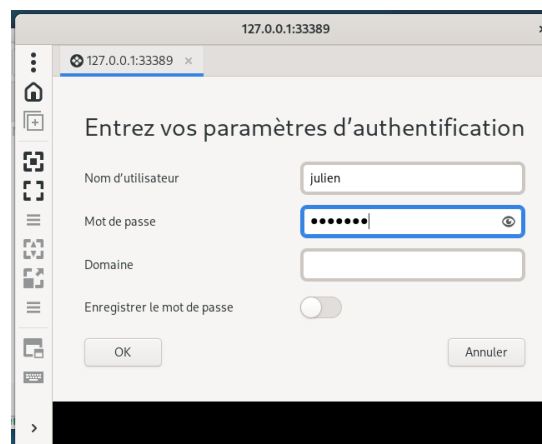
- **Protocole :** RDP (Remote Desktop Protocol).
- **Serveur :** 127.0.0.1:33389.



- *Note : On ne tape pas l'IP de la cible, mais l'entrée locale du tunnel créée sur le clone.*

Utilisateur : julien.

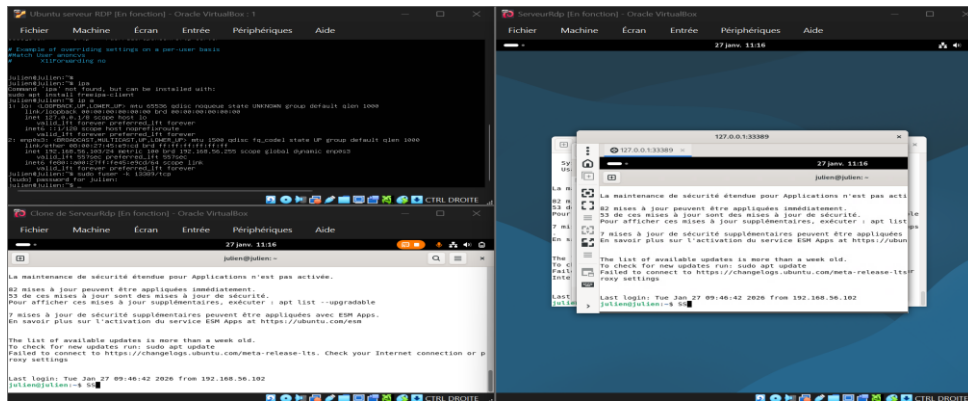
Mot de passe : Le mot de passe configuré dans les paramètres de partage de la cible.



Le mécanisme de transfert

Le succès de cette connexion prouve que le flux a parcouru le chemin suivant :

1. **Remmina** envoie les données sur le port 33389 de la machine locale (.104).
2. Le **Tunnel Local** transporte ces données vers le port 13389 de l'**Ubuntu Server** (.103).
3. Le **Tunnel Reverse** récupère ces données sur le port 13389 de l'Ubuntu pour les livrer au port 3389 de la **Debian Cible** (.102).



Conclusion Finale du TP

Le TP démontre la puissance des tunnels SSH pour créer des ponts sécurisés à travers des réseaux segmentés. L'utilisation combinée d'un tunnel local et d'un tunnel reverse permet de faire transiter un flux graphique (RDP) de manière totalement chiffrée et anonyme pour les machines d'extrémité, le tout centralisé sur un serveur de rebond Ubuntu.

1/ Créations d'une clé publique et privé

Dans l'objectif de sécuriser au max la connexion SSH à l'aide d'une documentation (<https://www.it-connect.fr/chapitres/authentification-ssh-par-cles/>)

Dans un premier temps sur votre debian13 graphique, dans un terminal taper les commandes suivantes :

Commande : ssh-keygen -b 4096

```
julien@ServeurRdp:~$ ssh-keygen -b 4096
```

Pour le reste vous pouvez mettre ce que vous voulez dans la passe phrase.

Commande : ssh-copy-id julien@192.168.56.103
(l'adresse IP de la Ubuntu pour cette exemple)

```
julien@ServeurRdp:~$ ssh-copy-id julien@192.168.56.103
```

Et si tout a bien marcher lorsqu'on refait le tunnel cela nous demande plus le mdp pour se connecter

2/OTP

Sur votre Ubuntu server, effectuer ses commandes

sudo apt update && install oathtool

- Par la suite, on génère une clé en base32 qui nous donne : (**IOTKZ25IG24FNPON**)

```
julien@julien:~$ head -c 10 /dev/urandom | base32
```

Nous avons créé un script qui intercepte ce que l'utilisateur tape et le compare au code calculé par oathtool.

- **Fichier :** **/usr/local/bin/check_otp.sh**

LES COMMANDES A METTRE DANS LE FICHIER :

```
#!/bin/bash  
  
read -r code  
  
SECRET="IOTKZ25IG24FNPON"  
  
attendu=$(/usr/bin/oathtool -b --totp "$SECRET")  
  
if [ "$code" == "$attendu" ]; then  
  
    exit 0  
  
else  
  
    exit 1  
  
fi
```

1. La Porte d'Entrée (Configuration SSH)

Nous avons dit à SSH qu'il ne doit plus se contenter de la clé SSH, mais qu'il doit **aussi** demander une interaction au clavier (le code).

- **Fichier :** **/etc/ssh/sshd_config**
- **Ligne ajoutée à la fin :**

AuthenticationMethods publickey,keyboard-interactive

2. Le Gardien (Configuration PAM)

C'est ici que nous avons fait le lien entre SSH et le script. Nous avons aussi désactivé le mot de passe habituel pour ne laisser passer que le code OTP.

- **Fichier :** **/etc/pam.d/sshd**

- **Modifications :**

1. On a désactivé le mot de passe système (ajout du #) : #@include common-auth
2. On lance le script en lui passant la saisie : auth required pam_exec.so expose_auth tok /usr/local/bin/check_otp.sh
3. On valide la connexion si le script est OK (pour éviter le crash "Connection Reset") : auth required pam_permit.so

Et voilà par la suite on se retrouve sur notre Debian graphique et vous tapez la commande `julien@IP_de_la_machine` et lorsque que l'on vous demande un mot de passe

3/KEEPASS

- **Mise en place pas-à-pas (Côté Client .104)**

Voici la procédure technique pour configurer ton "générateur de codes" personnel.

1) Installation

Sur ta machine Client pour (cet exemple la Debian), installe le logiciel :

```
sudo apt update && install keepassxc -y
```

2) Création du Coffre-fort

- Lance **KeePassXC**.
- Choisis "**Créer une nouvelle base de données**".
- Donne-lui un nom (ex: Pass_Projet_Lenoble).
- Définis un **Mot de passe Maître** fort.

3) Création de l'Entrée

- Clique sur le bouton + (Ajouter une entrée).
- Remplis les infos de base :
 1. **Titre** : Serveur Rebond SSH
 2. **Nom d'utilisateur** : julien
 3. **Mot de passe** : (Tu peux laisser vide ou mettre le mot de passe Linux, mais on ne s'en sert pas ici car on utilise des clés SSH).
 4. **URL** : 192.168.56.103

4) Configuration du TOTP

C'est ici que tu intègres la clé générée précédemment.

1. Fais un **Clic-Droit** sur ton entrée "Serveur Rebond SSH".
2. Va dans le menu **TOTP > Configurer le TOTP...**
3. Une fenêtre s'ouvre te demandant une "Clé secrète".
4. Copie-colle (ou tape) ta clé unique : **IOTKZ25IG24FNPON**. *(Les autres paramètres par défaut sont bons : RFC 6238, 30 secondes, 6 chiffres).*
5. Valide par **OK**.

Tu ouvres ton **KeePassXC** et tu le déverrouilles.

1. Tu lances ta commande SSH dans le terminal `ssh julien@192.168.56.103`
2. Le serveur demande Password:.
3. Tu vas dans KeePass, tu fais **Clic-Droit** sur l'entrée > **Afficher le TOTP** (ou Copier le TOTP).
4. Tu tapes ce code dans le terminal.
5. **Accès autorisé !**